
Mupix

Release 0.3.1

May 21, 2021

Contents

1	Installation	3
2	Usage	5
3	Features	7
4	Contents	9
4.1	License	9
4.2	Backward Compatibility	9
4.3	Python 2 Statement	9
4.4	How to contribute	10
4.5	Overview	10
4.6	Basic Objects	11
4.7	Commands	11
5	Indices and tables	13
	Python Module Index	15
	Index	17

CHAPTER 1

Installation

Build the latest version using poetry

- `poetry install`

Build the latest version using pip

- `pip install .`

Install it from PyPi

- `pip install mupix`

CHAPTER 2

Usage

For up-to-date usage information

- Checkout the [ReadTheDocs!](#)
- For the offline version run `mupix` or `mupix --help`

- Outputs valid JSON data
- Open and read (output to screen as JSON) symbolic music files
- Align musical markings from two or more symbolic files using sequence alignment algorithms
- Output the differences between two files as full error descriptions or as counted types.

– Error Description

```
{
  "ErrorDescription": {
    "part2_0.0_C major": "part2_0.0_G major",
    "part2_0.0_<music21.note.Note C>": "part2_0.0_<music21.note.Note G>",
    "part2_1.0_<music21.note.Note D>": "part2_1.0_<music21.note.Note A>",
    "part2_2.0_<music21.note.Note E>": "part2_2.0_<music21.note.Note B>",
    "part2_3.0_<music21.note.Note F>": "part2_3.0_<music21.note.Note C>",
    "part2_0.0_<music21.note.Note G>": "part2_0.0_<music21.note.Note D>",
    "part2_1.0_<music21.note.Note A>": "part2_1.0_<music21.note.Note E>",
    "part2_2.0_<music21.note.Note B>": "part2_2.0_<music21.note.Note F#>",
    "part2_3.0_<music21.note.Note C>": "part2_3.0_<music21.note.Note G>"
  }
}
```

– Counted error types

```
# Syntax highlighting as python because of the need for comments.
{
  "Notes": [
    {
      "right": 8,
      "wrong": 8,
      "name": "NoteNameResult"
    }, # example was truncated
    {
      "right": 111,
```

(continues on next page)

(continued from previous page)

```
        "wrong": 17,  
        "name": "NoteTotalResult"  
    }  
]  
}
```

4.1 License

Copyright © 2020 Alex Daigle

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

4.2 Backward Compatibility

This is very new software, and is prone to frequent breaking due to forming ideas and how the software should be structured.

It’s stance on compatibility is very much influenced by and . This should come into effect at version 1.0.0.

4.3 Python 2 Statement

Mupix is a Python3 package and there is no good reason to support Python2 at this time. If you believe you have a good reason, create an issue on GitHub.

4.4 How to contribute

This is very appreciated, thank you! Don't be afraid to post a half-written Pull-Request on .

4.5 Overview

This project wraps around *Music21* and *lxml* to make certain tasks easier and provide code for lining up and evaluating multiple symbolic music-files. I do not agree with certain design decisions *Music21* has taken regarding the handling of the MusicXML format. Issues which become observable to me when examining the output of multiple music engraving software.

4.5.1 Documentation

```
class mupix.application.BasicCompare (true_filepath: str, test_filepath: str, do_not_count: list
                                     = [])
```

Using 1-to-1 comparisons based on the index of each element. Obviously not ideal, but can be an interesting comparison.

```
class mupix.application.SimpleNeedlemanWunsch (true_filepath: str, test_filepath: str,
                                                do_not_count: list = [])
```

This version aligns notes based on the note names, and the measures + offsets of other elements.

Note: Align each note object with each other. Understandably this is not the best, but it serves more as a proof-of-concept or a placeholder to be built later.

- Notes are aligned by step names
 - Restes are aligned by measure number as a single char
 - TimeSignatures are aligned by measure number as a single char
 - KeySignatures are aligned by measure number as a single char
 - Clefs are aligned by measure number as a single char
-

```
class mupix.application.WeightedNeedlemanWunsch (true_filepath: str, test_filepath: str,
                                                do_not_count: list = [])
```

Using a weighted version of Affine Needleman-Wunsch, the way it should be used.

```
class mupix.application.PartwiseWeightedNeedlemanWunsch (true_filepath: str,
                                                         test_filepath: str,
                                                         do_not_count: list =
                                                         [])
```

Using weighted N-W on every permutation of parts between ground truths and test files. The ones with the lowest number of errors are aligned. Then a new total is computed with the lowest errored parts.

```
mupix.application.xml_validator (musicxml_filepath, schema_filepath='home/docs/checkouts/readthedocs.org/user_builds/mupix/packages/tests/xml/musicxml.xsd')
```

Return if the provided musicxml file is valid against the current musicxml schema.

Parameters

- **[musicxml_filepath]** (*String*) – A character string that represents the filepath and filename of the file to open.

- **[schema_filepath] (optional)** – A character string that represents the filepath and filename of the schema to use.

Returns Returns a boolean value, either it is a valid MusicXML file or it is not.

Return type Bool

`mupix.application.xml_type_finder(musicxml_filepath)`

Check if the xml file is written in a partwise or timewise fashion. This is just the fastest way to check as long as everything in the MusicXML isn't minified on a single line.

Parameters **[filepath]** (*String*) – A character string that represents the filepath and filename of the file to open.

Returns Either a Partwise or a Timewise string will return.

Return type String

4.6 Basic Objects

4.7 Commands

Mupix is available as a command line application. If all you want to do is compare files, there is no need to write any code. Just install it and run it from the command line. It should work on most platforms. **All commands return JSON data.**

4.7.1 Mupix Read

Mupix will read symbolic music-files and return json data with Notes, Rests, Time Signatures, Key Signatures, and, Clefs. You can even choose what data you want to read by using flags *-n*, *-r*, *-t*, *-k*, *-c*, or have the data be indented on output by using *-p*.

Example

You could read a file using the folloing:

```
$ mupix read ./my_musicxml_file.xml
```

You can *pretty-print* the text by using the *-p* option:

```
$ mupix -p read ./my_musicxml_file.xml
```

You can also use bash expansion for multiple files:

```
$ mupix -p read ./xml/*
```

4.7.2 Mupix Compare

Comparing two files using the command line. You can even choose what data you want to read by using flags *-n*, *-r*, *-t*, *-k*, *-c*, or have the data be indented on output by using *-p*. Or if you're just interested in the total, you can pass *-T*.

Example

You can compare two files using the following:

```
$ mupix compare ./ground_truth.xml ./5-D.xml
```

You can *pretty-print* the output:

```
$ mupix -p compare ./ground_truth.xml ./5-D.xml
```

You can change the Musical Marking alignment algorithm:

```
$ mupix compare --sort=anw-1 ./ground_truth.xml ./5-D.xml
```

You can choose what you wish to display, and combine commands together too:

```
$ mupix -nt compare ./ground_truth.xml ./5-D.xml
$ mupix -rk compare --sort=anw-1 ./ground_truth.xml ./5-D.xml
$ mupix -ntk compare --sort=basic ./ground_truth.xml ./5-D.xml
```

Just the total with *pretty-print*:

```
$ mupix -pT compare ./ground_truth.xml ./5-D.xml
```

You can even use bash expansion for multiple files to test against a single ground truth:

```
$ mupix -pT compare ./ground_truth.xml ./xml/*
```

4.7.3 Mupix Validate

You can check to see if the file is valid against the official MusicXML specification, this process can take ~20 seconds per file:

```
$ mupix validate ./testing_xml.xml
```

Or of multiple files:

```
$ mupix validate ./*
```

4.7.4 Mupix xml_finder

You can identify the type of MusicXML file, either part-wise or time-wise by using the *xml_finder* command:

```
$ mupix xml_finder ./testing_xml.xml
```

CHAPTER 5

Indices and tables

- `genindex`
- `modindex`
- `search`

m

`mupix.application`, [10](#)

`mupix.commands`, [11](#)

B

BasicCompare (*class in mupix.application*), [10](#)

M

mupix.application (*module*), [10](#)

mupix.commands (*module*), [11](#)

P

PartwiseWeightedNeedlemanWunsch (*class in mupix.application*), [10](#)

S

SimpleNeedlemanWunsch (*class in mupix.application*), [10](#)

W

WeightedNeedlemanWunsch (*class in mupix.application*), [10](#)

X

xml_type_finder() (*in module mupix.application*), [11](#)

xml_validator() (*in module mupix.application*), [10](#)